



Machine Learning and the Physical World

Lecture 8 : Reinforcement Learning

Carl Henrik Ek - che29@cam.ac.uk

5th of November, 2024

<http://carlhenrik.com>

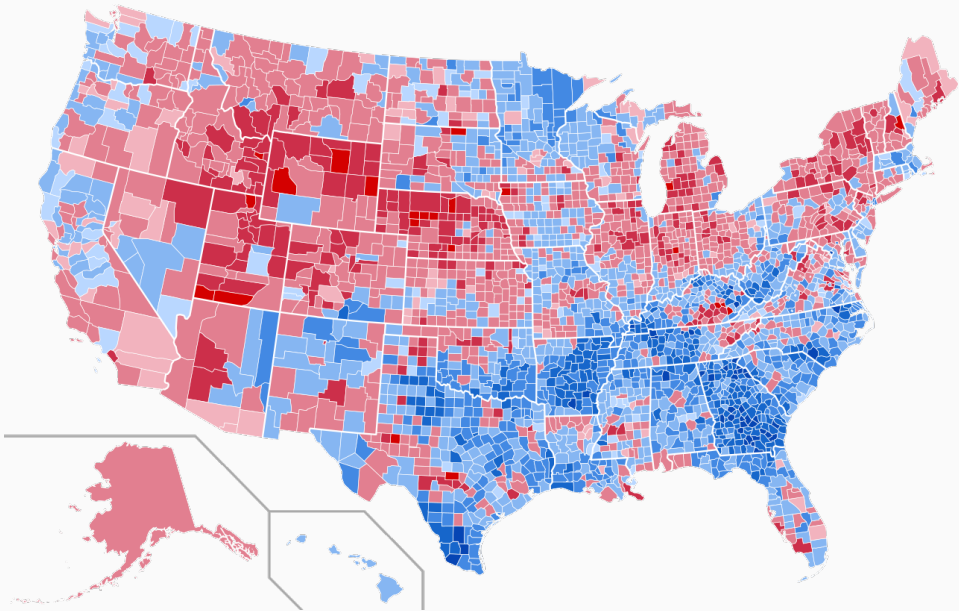
- The infeasibility of truth and the search for knowledge

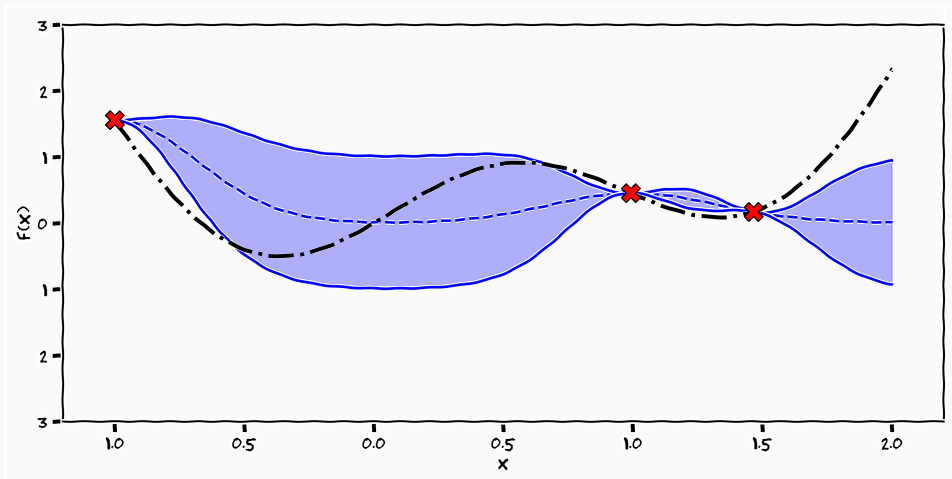
- The infeasibility of truth and the search for knowledge
- Parametrise our ignorance/beliefs

- The infeasibility of truth and the search for knowledge
- Parametrise our ignorance/beliefs
- Statistical Inference to update our knowledge from experiment

- The infeasibility of truth and the search for knowledge
- Parametrise our ignorance/beliefs
- Statistical Inference to update our knowledge from experiment
- Simulation and Emulation

- The infeasibility of truth and the search for knowledge
- Parametrise our ignorance/beliefs
- Statistical Inference to update our knowledge from experiment
- Simulation and Emulation
- Exploration using Emulators



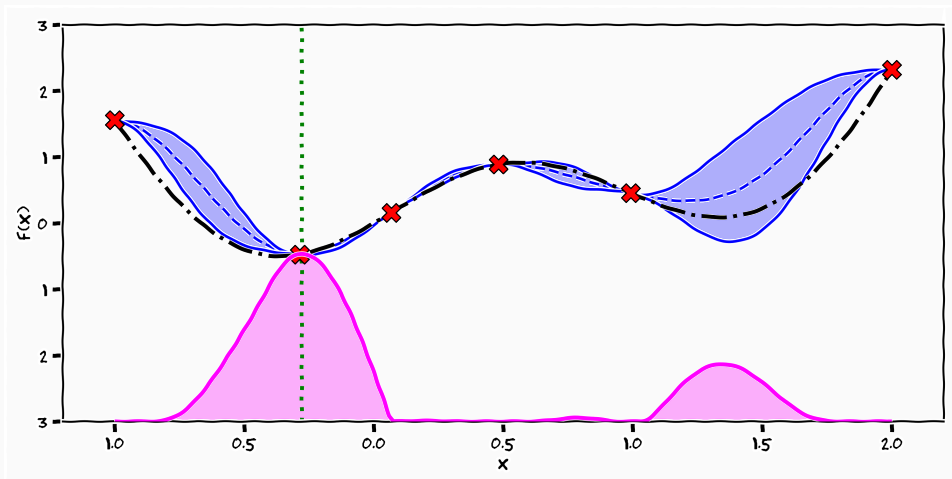


- Define a function that defines the **utility** of observing each location

$$u(x, f(x^{(*)}), \mathcal{M}_n)$$

- Define the acquisition function as the expected utility

$$\begin{aligned}\alpha(x; \{x_i, y_i\}_{i=1}^n, \mathcal{M}_n) &= \mathbb{E}_{p(f)}[u(x)] \\ &= \int u(x, f(x^{(*)}), \mathcal{M}_n) p(f \mid \{x_i, y_i\}_{i=1}^n) df\end{aligned}$$



November 5, 2024

Reinforcement Learning

Supervised Learning predict output from input

$$\mathcal{D} = \{y_i, x_i\}_{i=1}^N$$

$$p(\mathbf{y}|\mathbf{x}, \theta)$$

Supervised Learning predict output from input

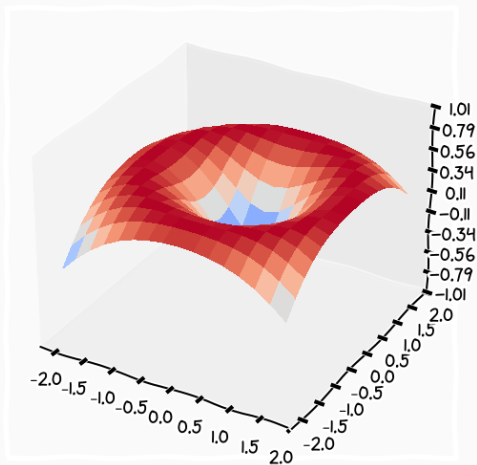
$$\mathcal{D} = \{y_i, x_i\}_{i=1}^N$$

$$p(\mathbf{y}|\mathbf{x}, \theta)$$

Unsupervised Learning model the data

$$\mathcal{D} = \{y_i\}_{i=1}^N$$

$$p(\mathbf{y}|\theta)$$



Reinforcement Training



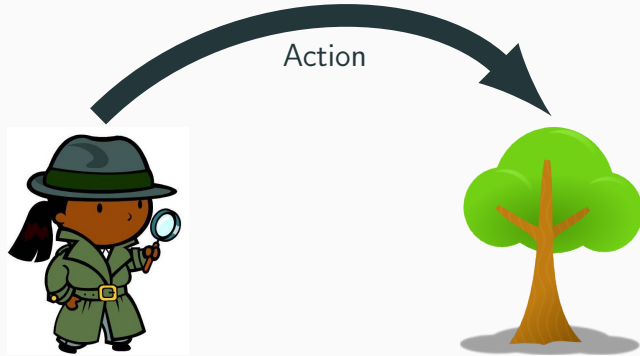
Reinforcement Learning

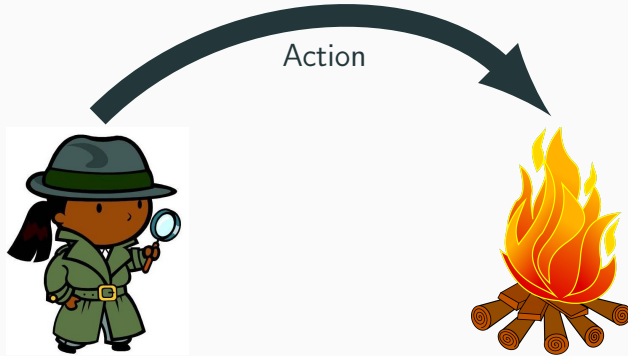


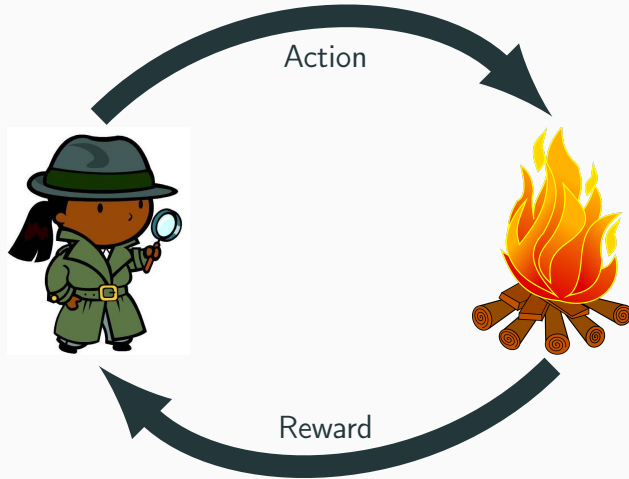
Can we learn without specifying how the task should be achieved by providing, rewards (positive) and punishment (negative)?



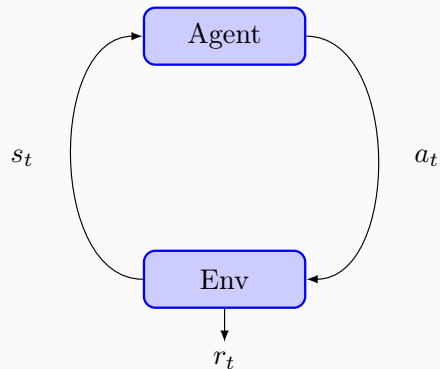


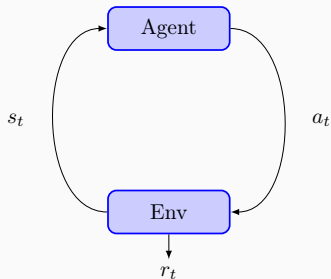






The Agent and the Environment





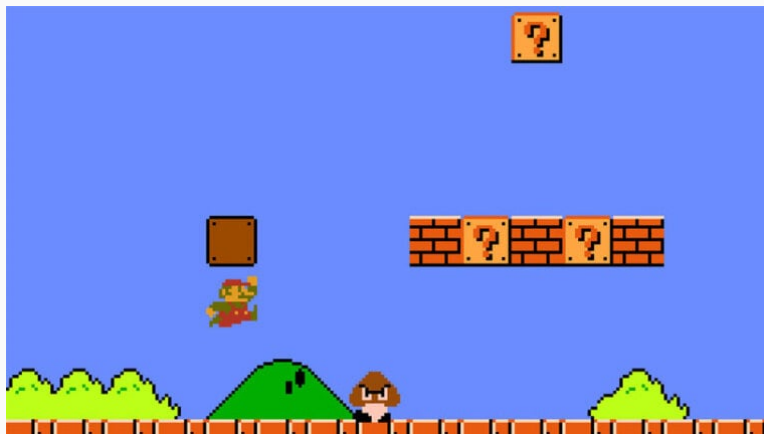
$$\operatorname{argmax}_{a_0, a_1, \dots} \sum_{t=0}^{\infty} r_t$$

- the agent takes actions in the environment
- actions change the state of the environment
- the state produces a reward

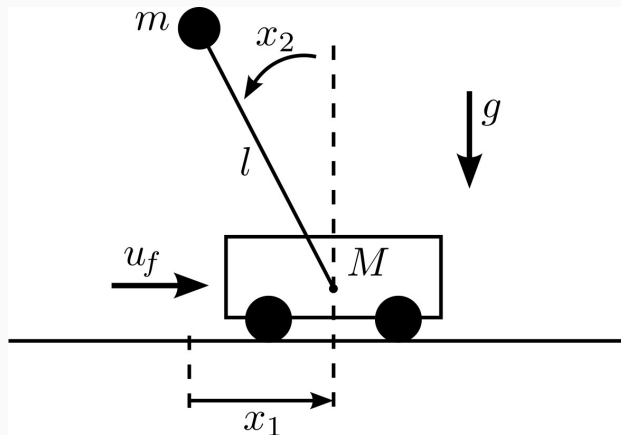
How is this different?

$$p\left(\sum_{t=0}^T r_t \mid s_0, a_0, a_1, \dots, a_t\right)$$

Lets just collect lots and lots of data and turn it into a regression problem



Inverted Pendulum



- When the number of possible sequences of input is very large

- When the number of possible sequences of input is very large
- When acquiring data is hard and expensive

- When the number of possible sequences of input is very large
- When acquiring data is hard and expensive
- When its hard to define the "structure" of the problem explicitly as a cost

Markov Decision Processes

Definition (Markov Decision Process)

A Markov Decision Process is a 4-tuple (S, A, T, R)

S is the State space

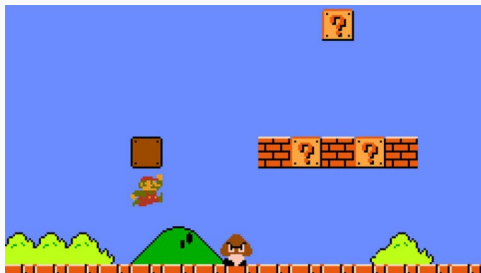
A is the Action space

T the state transition function

R the reward function

$$T(s_t, a_t) = T(s_t, a_t \mid s_{t-1}, a_{t-1})$$

- RL provides solutions to Markov Decision Processes
- We must define our state space in such a way that the environment forms an MDP



State Space

- position and velocity of Mario
- the score
- number of coins collected
- time remaining
- Goomba position & velocity, state
-



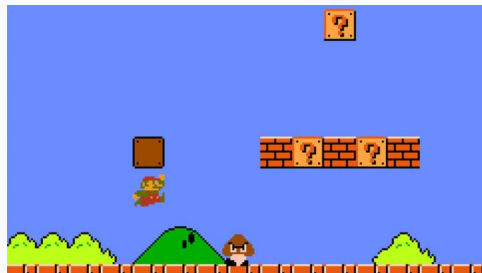
Action Space

- The controller has

$a, b, \uparrow, \downarrow, \leftarrow, \rightarrow$

- Can press certain buttons together

$$A = \underbrace{\{0, 1, \dots, 8\}}_{\text{directions}} \times \underbrace{\{0, 1, 2, 3\}}_{\text{buttons}}$$

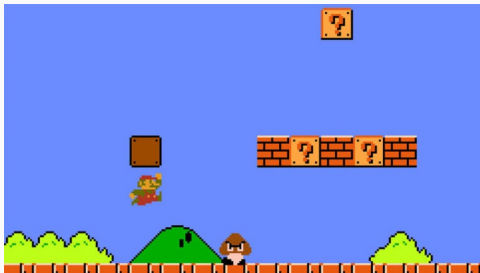


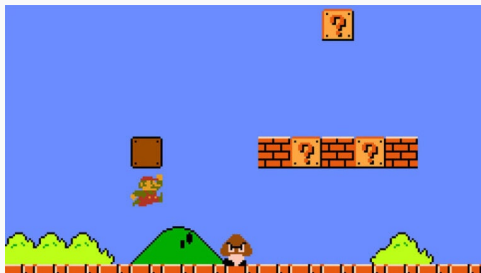
Transition Function

$$T : S \times A \rightarrow \Delta S$$

Reward Function

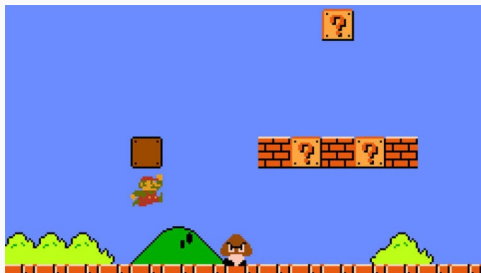
- beating the level (0, 1)





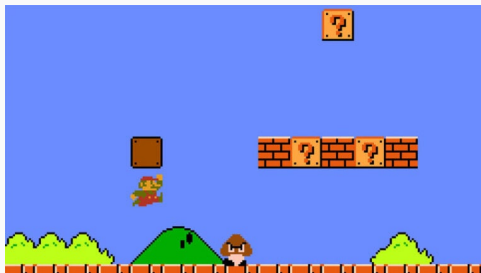
Reward Function

- beating the level (0, 1)
- total score



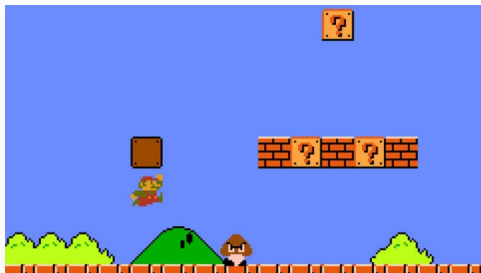
Reward Function

- beating the level $(0, 1)$
- total score
- beating the level $(0, 1) + 0.01 \times \text{score}$



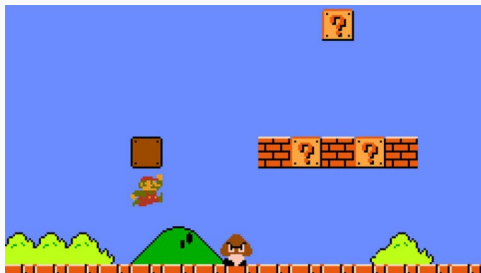
Reward Function

- beating the level $(0, 1)$
- total score
- beating the level
 $(0, 1) + 0.01 \times \text{score}$
- beating the level/time

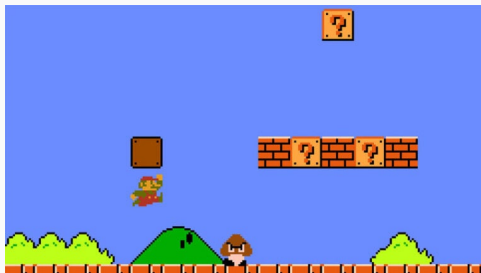


Reward Function

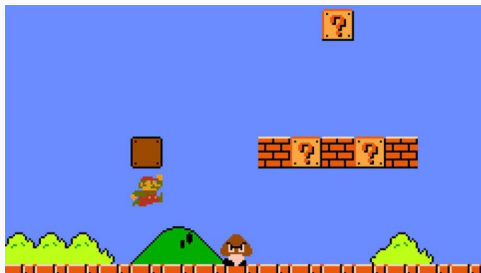
- beating the level $(0, 1)$
- total score
- beating the level
 $(0, 1) + 0.01 \times \text{score}$
- beating the level/time
-



- think carefully about constructing the environment



- think carefully about constructing the environment
- think carefully about how to structure reward signal



- think carefully about constructing the environment
- think carefully about how to structure reward signal
- now we have a problem to approach with RL

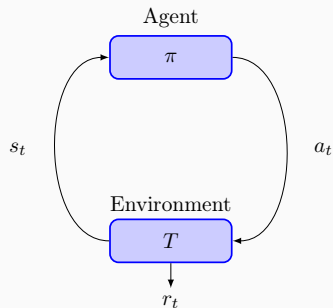
Reinforcement Learning

$$G = \sum_{t=0}^{\infty} r_t$$

- Task of RL is to design an agent to maximise return G

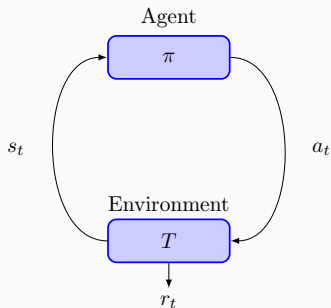
$$G = \sum_{t=0}^{\infty} \gamma^t r_t$$

- Task of RL is to design an agent to maximise return G
- We generally have a preference of cookies now compared to cookies later which we introduce by $\gamma \in (0, 1]$ called the discount factor



- The agent acts according to a policy π

$$\pi : S \rightarrow A$$



- The agent acts according to a policy π

$$\pi : S \rightarrow A$$

- Goal is to learn the optimal policy

$$\pi_* = \max_{\pi} \sum_{t=0}^{\infty} \gamma^t r_t$$

- We do not know the dynamics of the environment

$$p(s_{t+1} \mid s_t, a_t)$$

- We do not know the dynamics of the environment

$$p(s_{t+1} \mid s_t, a_t)$$

- We do not know the policy

$$p(a_t \mid s_t)$$

- We do not know the dynamics of the environment

$$p(s_{t+1} \mid s_t, a_t)$$

- We do not know the policy

$$p(a_t \mid s_t)$$

- We not might even know the reward

$$p(r_t \mid s_t)$$

- We do not know the dynamics of the environment

$$p(s_{t+1} \mid s_t, a_t)$$

- We do not know the policy

$$p(a_t \mid s_t)$$

- We not might even know the reward

$$p(r_t \mid s_t)$$

- We have no data to learn from

What do we want to learn?



- we want to interact with the environment as little as possible

What do we want to learn?



- we want to interact with the environment as little as possible
- we only want to know the dynamics up to a level where the policy will act

What do we want to learn?



- we want to interact with the environment as little as possible
- we only want to know the dynamics up to a level where the policy will act
- we only need to understand how to act to get a high reward

What do we want to learn?



- we want to interact with the environment as little as possible
- we only want to know the dynamics up to a level where the policy will act
- we only need to understand how to act to get a high reward
- learning the dynamic and policy are just proxies

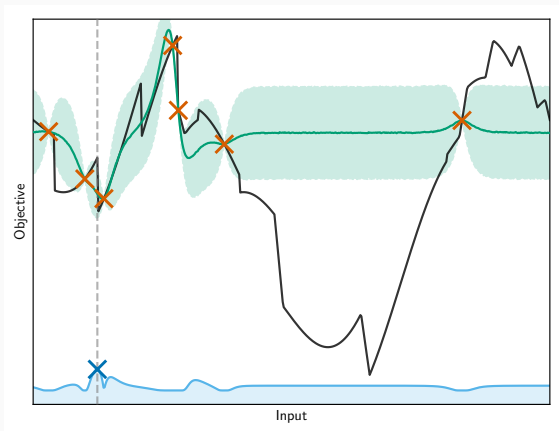
1. initially we do not have any data

1. initially we do not have any data
2. we usually start off with an initial experiment

1. initially we do not have any data
2. we usually start off with an initial experiment
3. when we get data we can update our dynamics and policy

1. initially we do not have any data
2. we usually start off with an initial experiment
3. when we get data we can update our dynamics and policy
4. how to acquire data from the knowledge that we have?

Bayesian Optimisation



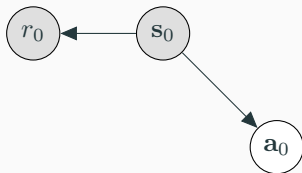
$$p(s_{t+1} \mid s_t, a_t)$$

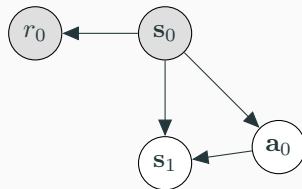
$$p(a_t \mid s_t)$$

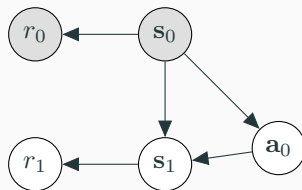
- Model the dynamics and the policy directly
- All the benefits of having uncertainty over each specific component
 - bias with prior knowledge
 - efficient exploration
- Just normal statistical inference :-)

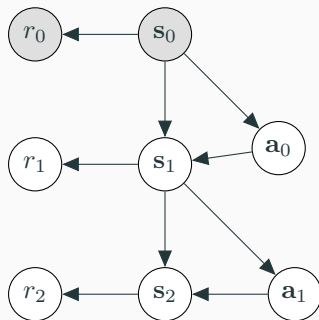


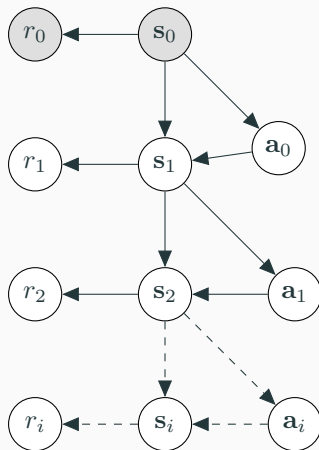


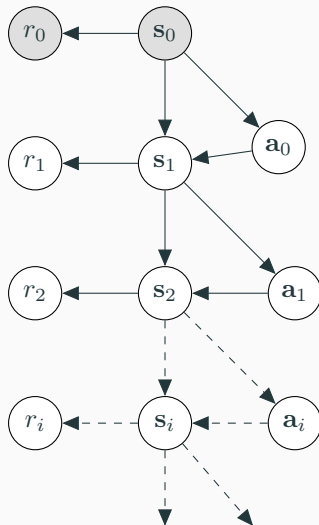




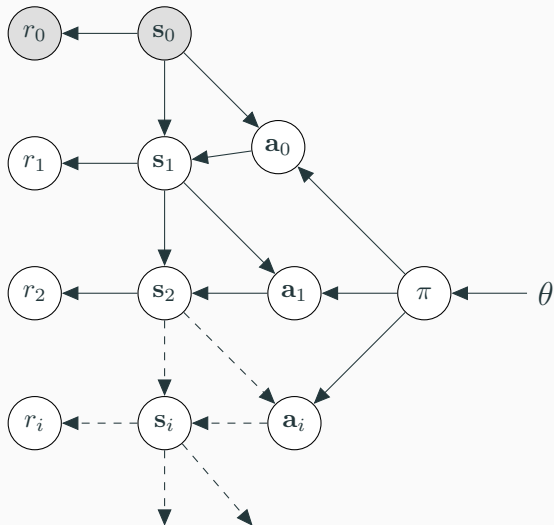




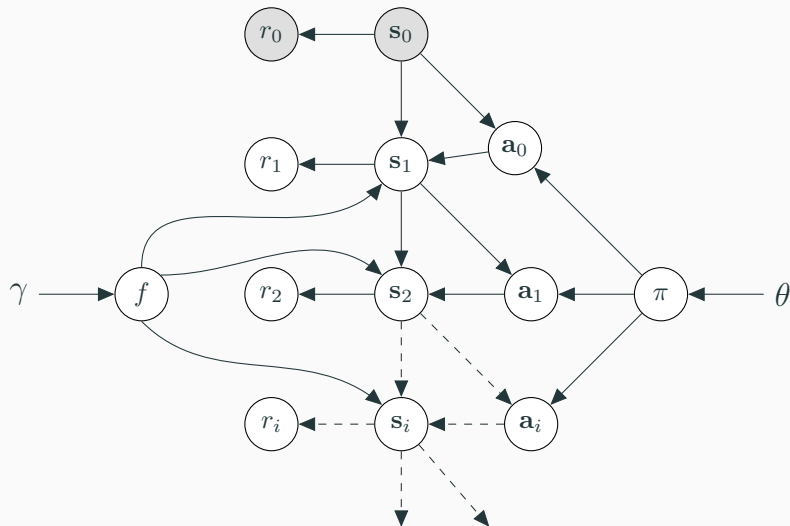




Reinforcement Learning



Reinforcement Learning



$$\begin{aligned}
p(\mathbf{s}_{1,\dots,T}, a_{0,\dots,T-1}, r_{0,\dots,T}, f, \pi, \gamma, \theta | \mathbf{s}_0) = & \\
p(\mathbf{s}_T | a_{T-1}, s_{T-1}, f) p(a_{T-1} | s_{T-1}, \pi) \dots & \\
\vdots & \\
p(\mathbf{s}_2 | a_1, s_1, f) p(a_1 | s_1, \pi) & \\
p(\mathbf{s}_1 | a_0, s_0, f) p(a_0 | s_0, \pi) & \\
p(f | \gamma) p(\pi | \theta) p(\gamma) p(\theta) &
\end{aligned}$$

- if we want to learn dynamics and policy we need to marginalise out f, π, θ and γ
- very very hard problem as uncertainty propagated a long way

$$\mathbb{E}[r_t] = \int \int p(r_t \mid s_{t+1}) p(s_{t+1} \mid s_t, a_t) p(a_t \mid s_t) da_t ds_{t+1}$$
$$\pi_* = \max_{\pi} \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t r_t \mid a_t \sim p(a_t \mid s_t) \right]$$

- We want to avoid explicitly learning a policy and a reward model
- Directly maximise the expected reward

- Define value function V ,

$$V_{\pi}(s_0) = \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t r_t \mid a_t \sim \pi(s_t) \right]$$

- Define value function V ,

$$V_{\pi}(s_0) = \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t r_t \mid a_t \sim \pi(s_t) \right]$$

- Lets move out the first action and define the function,

$$\tilde{V}_{\pi}(s_0, a_0) = \mathbb{E}[r_0 \mid a_0] + \mathbb{E} \left[\sum_{t=1}^{\infty} \gamma^t r_t \mid a_t \sim \pi(s_t) \right]$$

- Define value function V ,

$$V_{\pi}(s_0) = \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t r_t \mid a_t \sim \pi(s_t) \right]$$

- Lets move out the first action and define the function,

$$\tilde{V}_{\pi}(s_0, a_0) = \mathbb{E}[r_0 \mid a_0] + \mathbb{E} \left[\sum_{t=1}^{\infty} \gamma^t r_t \mid a_t \sim \pi(s_t) \right]$$

- This we will refer to as the Q -function,

$$Q_{\pi}(s_0, a_0) = \tilde{V}_{\pi}(s_0, a_0)$$

- How should we pick our policy?

- How should we pick our policy?
- The greedy policy is optimal (Bellman Optimality Equation)

$$\pi_*(s) = \operatorname{argmax}_{a \in A} Q_*(s, a)$$

- How should we pick our policy?
- The greedy policy is optimal (Bellman Optimality Equation)

$$\pi_*(s) = \operatorname{argmax}_{a \in A} Q_*(s, a)$$

- The optimal Q function,

$$Q_*(s, a) = \mathbb{E}[r_0 \mid a_0] + \mathbb{E} \left[\sum_{t=1}^{\infty} \gamma^t r_t \mid a_t \sim \pi_*(s_t) \right]$$

$$Q_*(s, a) = \mathbb{E}[r_0 \mid a_0] + \mathbb{E} \left[\sum_{t=1}^{\infty} \gamma^t r_t \mid a_t \sim \pi_*(s_t) \right]$$

Lets re-arrange things

$$\begin{aligned} Q_*(s, a) &= \mathbb{E}[r_0 \mid a_0] + \mathbb{E} \left[\sum_{t=1}^{\infty} \gamma^t r_t \mid a_t \sim \pi_*(s_t) \right] \\ &= \mathbb{E}[r_0 \mid a_0] + \gamma \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t r_{t+1} \mid a_{t+1} \sim \pi_*(s_{t+1}) \right] \end{aligned}$$

Lets re-arrange things

$$\begin{aligned} Q_*(s, a) &= \mathbb{E}[r_0 \mid a_0] + \mathbb{E} \left[\sum_{t=1}^{\infty} \gamma^t r_t \mid a_t \sim \pi_*(s_t) \right] \\ &= \mathbb{E}[r_0 \mid a_0] + \gamma \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t r_{t+1} \mid a_{t+1} \sim \pi_*(s_{t+1}) \right] \\ &= \mathbb{E}[r_0 \mid a_0] + \gamma \cdot V_*(s_1) \end{aligned}$$

Lets re-arrange things

$$\begin{aligned} Q_*(s, a) &= \mathbb{E}[r_0 \mid a_0] + \mathbb{E} \left[\sum_{t=1}^{\infty} \gamma^t r_t \mid a_t \sim \pi_*(s_t) \right] \\ &= \mathbb{E}[r_0 \mid a_0] + \gamma \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t r_{t+1} \mid a_{t+1} \sim \pi_*(s_{t+1}) \right] \\ &= \mathbb{E}[r_0 \mid a_0] + \gamma \cdot V_*(s_1) \\ &= \mathbb{E}[r_0 \mid a_0] + \gamma \cdot Q_*(s_1, \pi_*(s_1)) \end{aligned}$$

$$\begin{aligned} Q_*(s, a) &= \mathbb{E}[r_0 \mid a_0] + \mathbb{E} \left[\sum_{t=1}^{\infty} \gamma^t r_t \mid a_t \sim \pi_*(s_t) \right] \\ &= \mathbb{E}[r_0 \mid a_0] + \gamma \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t r_{t+1} \mid a_{t+1} \sim \pi_*(s_{t+1}) \right] \\ &= \mathbb{E}[r_0 \mid a_0] + \gamma \cdot V_*(s_1) \\ &= \mathbb{E}[r_0 \mid a_0] + \gamma \cdot Q_*(s_1, \pi_*(s_1)) \\ &= \mathbb{E}[r_0 \mid a_0] + \gamma \operatorname{argmax}_{a \in A} Q_*(s_1, a) \end{aligned}$$

$$\begin{aligned}\theta_* &= \operatorname{argmin}_{\theta} \left(Q_{\theta}(s, a) - \left(\mathbb{E}[r_0 \mid a_0] + \gamma \operatorname{argmax}_{a_1 \in A} Q_{\theta}(s_1, a) \right) \right)^2 \\ &= \operatorname{argmin}_{\theta} \left(\underbrace{\left(Q_{\theta}(s, a) - \gamma \operatorname{argmax}_{a_1 \in A} Q_{\theta}(s_1, a) \right)}_{\Delta Q} - \mathbb{E}[r_0 \mid a_0] \right)^2\end{aligned}$$

- Can we learn a θ such that we can approximate the optimal Q ?
- For any $\{s_0, s_1, a_0, r\}$ we will receive an error

- Same strategy as with BO but much harder to specify your utility

- Same strategy as with BO but much harder to specify your utility
- Acquisition functions to balance Exploration and Exploitation

- Same strategy as with BO but much harder to specify your utility
- Acquisition functions to balance Exploration and Exploitation
- Model based RL

- Same strategy as with BO but much harder to specify your utility
- Acquisition functions to balance Exploration and Exploitation
- Model based RL
 - we have parametrised our ignorance, how does this relate to the problem?

- Same strategy as with BO but much harder to specify your utility
- Acquisition functions to balance Exploration and Exploitation
- Model based RL
 - we have parametrised our ignorance, how does this relate to the problem?
- Model free RL

- Same strategy as with BO but much harder to specify your utility
- Acquisition functions to balance Exploration and Exploitation
- Model based RL
 - we have parametrised our ignorance, how does this relate to the problem?
- Model free RL
 - much harder

- Same strategy as with BO but much harder to specify your utility
- Acquisition functions to balance Exploration and Exploitation
- Model based RL
 - we have parametrised our ignorance, how does this relate to the problem?
- Model free RL
 - much harder
 - ϵ -greedy

- Same strategy as with BO but much harder to specify your utility
- Acquisition functions to balance Exploration and Exploitation
- Model based RL
 - we have parametrised our ignorance, how does this relate to the problem?
- Model free RL
 - much harder
 - ϵ -greedy
 - with probability $1 - \epsilon$ take the action that maximises the Q

- Same strategy as with BO but much harder to specify your utility
- Acquisition functions to balance Exploration and Exploitation
- Model based RL
 - we have parametrised our ignorance, how does this relate to the problem?
- Model free RL
 - much harder
 - ϵ -greedy
 - with probability $1 - \epsilon$ take the action that maximises the Q
 - with probability ϵ take a random action

- Model based RL
 - allows us to input our knowledge into the system
 - allows for (more) principled acquisition strategies
 - very challenging inference task

- Model based RL
 - allows us to input our knowledge into the system
 - allows for (more) principled acquisition strategies
 - very challenging inference task
- Model free RL
 - simple to implement and relatively cheap
 - hard to device principle acquisition strategies
 - often very data inefficient

Summary

- Reinforcement learning is often explained as a thing in itself

- Reinforcement learning is often explained as a thing in itself
- It is not, its just a very specific, recursive probabilistic model

- Reinforcement learning is often explained as a thing in itself
- It is not, its just a very specific, recursive probabilistic model
- We apply RL to problems where it is infeasible to acquire all data at the same time

- Reinforcement learning is often explained as a thing in itself
- It is not, its just a very specific, recursive probabilistic model
- We apply RL to problems where it is infeasible to acquire all data at the same time
- Just as BO the data-acquisition therefore has to be part of the inference mechanism

- Reinforcement learning is often explained as a thing in itself
- It is not, its just a very specific, recursive probabilistic model
- We apply RL to problems where it is infeasible to acquire all data at the same time
- Just as BO the data-acquisition therefore has to be part of the inference mechanism
- There are many things where RL is a suitable approach

- RL raises interesting general

- RL raises interesting general
- Finite capacity of models

- RL raises interesting general
- Finite capacity of models
 - How do we forget information no longer relevant?

- RL raises interesting general
- Finite capacity of models
 - How do we forget information no longer relevant?
 - Requires an understanding of my own knowledge

- RL raises interesting general
- Finite capacity of models
 - How do we forget information no longer relevant?
 - Requires an understanding of my own knowledge
- Proxy objectives

- RL raises interesting general
- Finite capacity of models
 - How do we forget information no longer relevant?
 - Requires an understanding of my own knowledge
- Proxy objectives
 - just as BO, learning the dynamic and the policy are just proxies to maximise the reward

- RL raises interesting general
- Finite capacity of models
 - How do we forget information no longer relevant?
 - Requires an understanding of my own knowledge
- Proxy objectives
 - just as BO, learning the dynamic and the policy are just proxies to maximise the reward
- Hierarchical decomposition of uncertainty

- RL raises interesting general
- Finite capacity of models
 - How do we forget information no longer relevant?
 - Requires an understanding of my own knowledge
- Proxy objectives
 - just as BO, learning the dynamic and the policy are just proxies to maximise the reward
- Hierarchical decomposition of uncertainty
 - am I uncertain of the reward because I don't know what I should do (policy) but if I act I know exactly what happens (dynamic)?

- RL raises interesting general
- Finite capacity of models
 - How do we forget information no longer relevant?
 - Requires an understanding of my own knowledge
- Proxy objectives
 - just as BO, learning the dynamic and the policy are just proxies to maximise the reward
- Hierarchical decomposition of uncertainty
 - am I uncertain of the reward because I don't know what I should do (policy) but if I act I know exactly what happens (dynamic)?
 - or am I uncertain of the reward because I know what I should do (policy) but I don't know what will happen if I do (dynamic)?

- Everyone loves RL till they start working on it

- Everyone loves RL till they start working on it
- RL has very many moving parts making it hard to understand the effects of different components

- Everyone loves RL till they start working on it
- RL has very many moving parts making it hard to understand the effects of different components
- The literature is good (Sutton et al., [1998](#)) and bad arXiv

- Everyone loves RL till they start working on it
- RL has very many moving parts making it hard to understand the effects of different components
- The literature is good (Sutton et al., [1998](#)) and bad arXiv
- RL is very hard meaning its as much an art as a science to get it to work

- Everyone loves RL till they start working on it
- RL has very many moving parts making it hard to understand the effects of different components
- The literature is good (Sutton et al., [1998](#)) and bad arXiv
- RL is very hard meaning its as much an art as a science to get it to work
- Don't do RL for your project!

- Everyone loves RL till they start working on it
- RL has very many moving parts making it hard to understand the effects of different components
- The literature is good (Sutton et al., [1998](#)) and bad arXiv
- RL is very hard meaning its as much an art as a science to get it to work
- Don't do RL for your project!

• Please don't

eof

References

-  Sutton, Richard S. and Andrew G. Barto (1998). *Introduction to Reinforcement Learning*. 1st. Cambridge, MA, USA: MIT Press.